

battery-rush-arena-script

슬라이드 1 — 표지 (Battery Rush Arena) 🕒 약 20초

안녕하세요. 게임 네트워크 기초 중간 프로젝트 발표를 맡은 [이름]입니다.

제가 이번에 구현한 프로젝트는 실시간 2인 배틀 게임인 '**Battery Rush Arena**' 입니다.

Unity 6000.3.10f1 클라이언트와 C# .NET 10 기반의 서버, 그리고 MySQL 8.x 버전을 연동하여 구축했습니다.

특히 **모든 게임 로직을 서버에서 판정하는 권위 서버 아키텍처**를 채택하고, 경기 결과를 데이터베이스에 안전하게 저장하도록 설계했습니다.

슬라이드 2 — 목차 🕒 약 25초

오늘 발표는 총 10개 섹션으로 구성했습니다.

[→ 화면 가리키며] 프로젝트 개요와 시스템 구조를 시작으로, 서버 컴포넌트와 프로토콜 설계를 살펴본 뒤, 실제 게임 로직인 이동, 배터리, 함정 시스템에 대해 설명하겠습니다.

마지막으로 MySQL을 활용한 DB 스키마와 저장 흐름, 그리고 Unity 클라이언트 구조를 정리하며 데모 시연으로 마무리하겠습니다.

슬라이드 3 — 프로젝트 개요 🕒 약 50초

이번 프로젝트의 핵심은 '**권위 서버(Authoritative Server)**' 방식입니다.

[→ 왼쪽 텍스트] Unity 클라이언트는 오직 사용자의 입력만 서버로 보내고, 서버에서 계산된 결과 화면을 렌더링하는 역할만 수행합니다. **모든 판정은 서버에서 일어납니다.**

서버는 TCP **포트 7777**을 사용하며, '**bra-spike-v1**' 이라는 전용 프로토콜 버전을 통해 통신합니다.

매치 시간은 **120초**로 설정되어 있습니다.

[→ 우측 기술 카드 3개] 기술 스택으로는 렌더링을 담당하는 Unity, 로직과 DB 관리를 담당하는 .NET 서버, 그리고 전적 저장을 위한 MySQL 8.x를 사용했습니다.

슬라이드 4 — 전체 시스템 구조 🕒 약 60초

전체적인 데이터 흐름을 살펴보겠습니다.

[→ 왼쪽에서 오른쪽 화살표] 클라이언트가 **입력 프레임**을 전송하면, C# 서버가 이를 받아 게임 상태를 갱신하고 다시 클라이언트로 브로드캐스트합니다.

경기가 종료되면 서버는 MySQL에 데이터를 **INSERT** 또는 **UPSERT** 합니다.

통신의 안정성을 위해 몇 가지 파라미터를 설정했는데요.

[→ 하단 파라미터 영역] **7777번 포트**를 사용하고, **2초 간격으로 하트비트**를 주고받습니다. 만약 **30초 동안 응답이 없는** 클라이언트는 연결이 끊긴 것으로 간주하는 **StaleTimeout** 로직을 적용했습니다.

슬라이드 5 — 서버 컴포넌트 구조 약 2분 30초

서버는 총 **10개의 핵심 컴포넌트**로 역할을 엄격하게 분리했습니다. 각각 어떤 일을 하는지 짚어보겠습니다.

[→ 카드 1: Program.cs] **Program.cs**는 진입점이자 의존성 조립소입니다.

`SpikeServerConfig.CreateDefault()` 로 설정 객체를 만들고, `RoomRegistry` 와 `MySQLPersistenceService` 를 생성해 `SpikeServerHost` 에 주입합니다. 이후 `Ctrl+C` 를 처리하는 취소 토큰을 연결하고 `host.RunAsync()` 로 서버를 시작합니다. 역할은 딱 하나, **의존성 조립**입니다.

[→ 카드 2: SpikeServerHost] **SpikeServerHost**는 서버의 핵심 엔진입니다. TCP 7777 포트에서 클라이언트를 수락하면 연결마다 `HandleClientAsync` 를 `Task.Run` 으로 독립 실행합니다. `hello` · `create_room` · `input_frame` 등 모든 메시지 라우팅이 여기서 이루어지고, `MonitorRoomsAsync` 가 **250ms 주기**로 Countdown 진행·배터리 재생성·매치 종료 같은 시간 기반 이벤트를 처리합니다.

[→ 카드 3: ClientSession] **ClientSession**은 TCP 연결 하나당 생성되는 상태 컨테이너입니다. `Guid` 로 발급한 세션 ID, 플레이어 이름, 소속 룸 코드를 보유합니다. `LastSeenUtc` 는 **30초 무응답 탐지**에, `LastProcessedTick` 은 **중복·오래된 입력 프레임 거부**에 사용됩니다. 예를 들어 네트워크 지연으로 같은 틱 번호의 패킷이 두 번 도착하면 두 번째는 즉시 `input_frame_ignored` 로 거부됩니다.

[→ 카드 4: RoomRegistry] **RoomRegistry**는 인메모리 룸 디렉토리입니다. 방 생성 시마다 `ROOM01` · `ROOM02` 형태의 코드를 자동 채번하고, 조회·참가·퇴장 연산 전체를 `SyncRoot` 락으로 스레드 안전하게 보호합니다. 호스트가 나가면 첫 번째 남은 멤버가 자동으로 호스트가 됩니다.

[→ 카드 5: SpikeRoom] **SpikeRoom**은 한 방의 모든 권위 상태를 담은 데이터 컨테이너입니다. 플레이어 위치·점수, 배터리·함정 좌표, 슬로우 효과 만료 시각, 슬로우샷 쿨다운이 이 객체 하나에 집중됩니다. 룸 상태는 `Lobby` → `Countdown` → `Active` → `Ended` → `Saving` → `ResultsReady` **6단계 열거형**으로 전이되며, 현재 단계에 따라 허용되는 동작이 달라집니다.

[→ 카드 6: LengthPrefixedProtocol] **LengthPrefixedProtocol**은 모든 네트워크 I/O의 단일 창구입니다. 송신 시 **4바이트 big-endian 길이 헤더**를 붙여 TCP 스트림의 메시지 경계를 명확히 구분합니다. 스트림마다 `SemaphoreSlim(1,1)` 을 `ConditionalWeakTable` 로 걸어 동시 쓰기를 직렬화함으로써 **패킷 인터리빙 현상**을 원천 차단합니다.

[→ 카드 7: ProtocolMessages] **ProtocolMessages**는 통신 봉투 클래스입니다.

`ClientMessage`에는 이동 벡터 `MoveX·MoveY`, 틱 번호, 스킬 여부가 담기고, `ServerMessage`에는 룸 스냅샷 전체가 담깁니다. JSON 옵션에 `WhenWritingDefault`를 적용해 기본값 필드는 전송하지 않아 **패킷 크기를 최소화**합니다.

[→ 카드 8: MySqlPersistenceService] **MySqlPersistenceService**는 경기 종료 후 MySQL에 결과를 저장합니다. `match_results` INSERT와 `player_stats` UPSERT를 **단일 트랜잭션**으로 묶고, 실패 시 `250ms × 시도 횟수` 만큼 대기 후 **최대 3회** 재시도합니다. 성공하면 즉시 상위 10명 리더보드를 쿼리해 반환합니다.

[→ 카드 9: SpikeServerConfig] **SpikeServerConfig**는 서버 전체 상수의 **단일 출처**입니다. 포트 **7777**, 매치 **120초**, 배터리 **3개**, 함정 **4개**, 슬로우샷 쿨타임 **4초** 등 수십 개의 값이 `CreateDefault()` 팩토리 하나에 집중됩니다. 게임 밸런스를 조정할 때 이 파일 하나만 수정하면 전체에 반영됩니다.

[→ 카드 10: PersistenceModels] **PersistenceModels**는 저장 계층의 DTO 모음입니다. `MatchResultPayload`는 매치 ID·종료 사유·승자·각 플레이어 점수를 담아 MySQL로 전달되고, `PersistenceAttemptResult`는 저장 성공 여부와 리더보드 스냅샷을 `SpikeServerHost`에 돌려줍니다.

슬라이드 5.1 — 컴포넌트 통신 흐름도 🕒 약 70초

각 컴포넌트의 역할을 파악했으니, 이번엔 이들이 **실시간으로 어떻게 연결되어 패킷 하나를 처리하는지** 전체 흐름을 보겠습니다.

[→ 왼쪽 의존 관계 트리] `Program.cs`가 `SpikeServerHost`를 생성하고, `SpikeServerHost`가 `RoomRegistry · LengthPrefixedProtocol · MySqlPersistenceService` 세 컴포넌트를 직접 보유합니다. `RoomRegistry` 아래엔 `SpikeRoom · ClientSession`이, `LengthPrefixedProtocol` 아래엔 `ProtocolMessages`가, `MySqlPersistenceService` 아래엔 `PersistenceModels`가 의존합니다. 맨 아래 초록 바인 `SpikeServerConfig`는 모든 컴포넌트가 공통으로 참조하는 설정값입니다.

[→ 오른쪽 5단계 흐름] `input_frame` 패킷 하나가 처리되는 경로를 따라가 보겠습니다. ① **수신**: `SpikeServerHost.HandleClientAsync`가 TCP 스트림에서 바이트를 읽습니다. ② **역직렬화**: `LengthPrefixedProtocol.ReadAsync`가 4바이트 헤더를 읽고 JSON 본문을 `ClientMessage`로 변환합니다. ③ **라우팅**: `message.Type`이 `input_frame`이면 `HandleInputFrameAsync`로 분기됩니다. ④ **로직 적용**: `RoomRegistry`와 `SpikeRoom`이 플레이어 위치·배터리 획득·함정 효과를 갱신합니다. ⑤ **브로드캐스트**: `LengthPrefixedProtocol.WriteAsync`가 갱신된 `room_snapshot`을 접속 중인 모든 클라이언트에 전송합니다.

각 단계가 명확히 분리된 컴포넌트를 통과하기 때문에, 버그 발생 시 어느 단계에서 문제가 생겼는지 즉시 추적할 수 있습니다.

슬라이드 6 — 프로토콜 설계 🕒 약 80초

효율적이고 명확한 통신을 위해 패킷 구조를 최적화했습니다.

[→ 상단 패킷 구조 다이어그램] 패킷은 **4바이트 BigEndian int32 길이** 헤더 뒤에 **UTF-8 JSON 페이로드**를 붙이는 구조입니다. BigEndian을 선택한 이유는 네트워크 바이트 순서의 표준이 BigEndian이기 때문입니다. 양쪽 장비의 엔디안이 달라도 명시적 변환으로 일치가 보장됩니다. 오른쪽 코드 블록처럼 실제 페이로드는 `{ "type": "input_frame", "dx": 1, "dy": 0 }` 형태의 compact JSON 한 줄입니다.

[→ 설계 결정 카드 1: SemaphoreSlim(1,1)] 첫 번째 결정은 **스트림당 SemaphoreSlim(1,1)** 입니다.

`MonitorRoomsAsync`의 브로드캐스트와 입력 처리 ACK가 같은 클라이언트에 동시에 전송되는 상황을 생각해보겠습니다. 락 없이 두 `WriteAsync`가 겹치면 패킷 A의 헤더 뒤에 패킷 B의 페이로드가 끼어드는 **인터리빙 현상**이 발생합니다.

`SemaphoreSlim(1,1)`은 이진 세마포어로 동작해 `WriteAsync`를 한 번에 하나씩만 실행하도록 직렬화하고, `ConditionalWeakTable`로 스트림 생명주기에 락을 자동으로 묶습니다.

[→ 설계 결정 카드 2: CaseInsensitive JSON 파싱] 두 번째는 **대소문자 무감지 JSON 파싱**입니다. Unity 클라이언트 쪽 코드와 C# 서버의 JSON 라이브러리가 필드명 규칙에 미묘한 차이가 생길 수 있습니다. 예를 들어 서버가 `roomCode`로 직렬화했는데 클라이언트가 `RoomCode`로 파싱을 시도하면 값이 null로 떨어집니다. `PropertyNameCaseInsensitive = true` 설정 한 줄로 이 문제를 원천 차단합니다.

[→ 설계 결정 카드 3: WriteIndented = false] 세 번째는 **compact JSON 직렬화**입니다. pretty-print는 들여쓰기와 줄바꿈이 추가되어 동일한 데이터라도 패킷 크기가 2~3배 커집니다. `input_frame`은 초당 수십 번 전송되는 고빈도 메시지이므로 이 차이가 누적 트래픽에 직결됩니다. `WriteIndented = false`가 기본값이고, 디버깅이 필요할 때만 이 옵션을 `true`로 바꾸면 즉시 pretty-print로 전환됩니다.

슬라이드 7 — 메시지 타입 🕒 약 40초

클라이언트와 서버가 주고받는 메시지는 명확히 정의되어 있습니다.

[→ 왼쪽 C→S 목록] 클라이언트는 hello, 방 생성, 준비 상태, 그리고 초당 여러 번 전송되는 `input_frame` 등 **8가지 메시지**를 보냅니다.

[→ 오른쪽 S→C 목록] 이에 대해 서버는 수락 여부나 룸 스냅샷, 카운트다운, 게임 중 배터리 재생성이나 효과 변경 알림 등 **총 16가지의 세분화된 메시지**로 응답하며 게임 상태를 정밀하게 동기화합니다.

슬라이드 8 — 룸 상태 머신 🕒 약 65초

게임의 흐름은 **6단계의 상태 머신**으로 제어됩니다.

[→ 상단 상태 전이도] 처음 **Lobby** 에서 두 명의 플레이어가 모두 준비되고 호스트가 시작 버튼을 누르면 **3초간의 Countdown** 으로 넘어갑니다.

이후 **120초 동안 Active 상태**에서 실제 배틀이 진행됩니다.

경기가 끝나거나 플레이어가 나가면 **Ended** 상태가 되고, 서버는 즉시 **Saving** 상태로 진입하여 MySQL 트랜잭션을 시작합니다.

저장이 완료되면 **ResultsReady** 상태가 되며 리더보드와 최종 결과가 클라이언트에 전달됩니다.

[→ 하단 4개 상세 카드] 이 상태들 중 특히 중요한 네 단계의 세부 동작을 아래에 정리했습니다.

슬라이드 9 — 게임 로직 ① 이동 & 배터리 🕒 약 45초

이제 실제 게임 규칙입니다. 우선 **이동 시스템**입니다.

[→ 왼쪽 파라미터 테이블] 플레이어는 프레임당 **0.75 유닛**씩 이동하며, 경기장은 **±7.25 유닛** 범위를 가집니다. 스폰 위치는 각각 좌표 (-5.1, 0) 과 (5.1, 0) 으로 서로 마주보는 구조입니다.

[→ 오른쪽 파라미터 테이블] **배터리**는 경기장 내에 동시에 **3개**가 유지되도록 서버가 랜덤한 위치에 생성합니다. 플레이어가 **0.65 유닛** 안으로 접근하면 획득 판정이 나며 **1점**을 얻습니다. 획득된 배터리는 **3초 후** 재생성되고, 서버는 전체 클라이언트에 `battery_respawned` 이벤트를 브로드캐스트합니다.

슬라이드 10 — 게임 로직 ② 함정 & SlowShot 🕒 약 45초

단조로움을 피하기 위해 **함정과 공격 요소**를 추가했습니다.

[→ 왼쪽 파라미터 테이블] 맵에는 **4개의 함정**이 상주하며, 반경 **0.65 유닛** 안에 들어오면 **0.75초 동안 이동 속도가 70%** 로 감소합니다.

[→ 오른쪽 파라미터 테이블] 플레이어는 **SlowShot 스킬**을 사용할 수 있습니다. 사거리는 **3.5 유닛**이고, 상대 방향과의 내적 값이 **0.25 이상**일 때 적중한 것으로 판정합니다. 맞은 상대는 **1.25초 동안 이동 속도가 65%** 로 줄어듭니다. 스킬 쿨타임은 **4초**이며, 남용을 막기 위해 서버가 이 쿨타임을 엄격하게 권위적으로 관리합니다.

슬라이드 11 — Input Frame 처리 흐름 🕒 약 65초

가장 빈번하게 일어나는 **입력 처리 흐름**을 스웬라인으로 정리했습니다.

[→ Unity Client 레인] 먼저 Unity 클라이언트가 키 입력을 감지하면 `dx, dy` 이동 값과

SlowShot 여부를 담아 **SendInputFrameAsync** 를 호출합니다. LengthPrefixed Protocol을 통해 직렬화된 패킷이 서버로 전송됩니다.

[→ SpikeServerHost 레인] 서버의 Host가 이 패킷을 받으면, 해당 플레이어의 **마지막 처리 틱 번호와 대조**하여 유효한 프레임인지 검증합니다. 중복되거나 오래된 프레임은 **input_frame_ignored** 로 즉시 거부됩니다.

[→ SpikeRoom 레인] 검증을 통과하면 **ApplyInput** 로직이 실행되어 위치, 배터리, 함정 상태를 갱신합니다. 서버는 갱신된 전체 게임 상태를 **active_tick** 메시지에 담아 브로드캐스트하고, 클라이언트는 이를 받아 메인 스레드에서 화면을 업데이트합니다.

슬라이드 12 — DB 스키마 약 90초

전적 저장을 위한 데이터베이스 구조는 **테이블 두 개**로 구성됩니다. 먼저 데이터베이스 자체를 **utf8mb4** 문자셋으로 선언했습니다. MySQL의 기본 **utf8** 은 사실 3바이트만 지원해 이보다 지나 일부 특수문자를 저장하지 못하는 문제가 있습니다. **utf8mb4** 는 완전한 4바이트 UTF-8로, 한국어 플레이어 이름을 포함한 모든 유니코드 문자를 안전하게 저장합니다.

[→ match_results 테이블] 컬럼 타입 선택 이유를 짚어보겠습니다. **match_id** 는 **VARCHAR(64)** PK입니다. 서버에서 **Guid.NewGuid().ToString("N")** 으로 생성하는 32자 hex 문자열을 키로 쓰며, 64자 여유를 준 것은 향후 포맷 변경 가능성을 고려한 것입니다.

ended_at_utc 는 **DATETIME(6)** 으로 **마이크로초 정밀도**까지 저장합니다. .NET의 **DateTimeOffset.UtcNow** 가 틱 단위 정밀도를 갖기 때문에, MySQL에서 가장 높은 정밀도인 **DATETIME(6)**을 맞춰 손실 없이 기록합니다. **winner_player_name** 이 **NULL 허용**인 이유는 무승부 종료 시 승자가 없기 때문입니다. **player_b_name** 과 **player_b_score** 도 NULL 허용인데, ServerAbort처럼 상대방이 연결되지 않은 채 비정상 종료되는 케이스를 처리하기 위해서입니다. 마지막으로 **raw_payload_json** 은 MySQL 네이티브 **JSON 타입**으로 매치 전체 결과 객체를 그대로 백업합니다. 나중에 분석 컬럼이 필요해도 스키마를 변경하지 않고 JSON 안에 새 필드를 추가하면 되므로 확장이 자유롭습니다.

[→ player_stats 테이블] **player_name** 을 **VARCHAR(64)** PK로 사용합니다. 서버가 **hello** 핸드셰이크 시점에 중복 이름을 **#2**, **#3** 접미사로 강제 해소하기 때문에, 이름 자체가 곧 고유 식별자가 됩니다. **wins**, **draws**, **losses**, **total_matches** 는 모두 **INT NOT NULL DEFAULT 0** 으로 선언했습니다. **DEFAULT 0**은 플레이어의 첫 경기 UPSERT 시 별도 초기화 쿼리 없이 바로 누적 시작되게 하기 위해서입니다. **best_score** 도 **DEFAULT 0**으로 시작하며, UPSERT 시 **GREATEST(best_score, 새 점수)** 로 최고 기록을 갱신합니다.

[→ 하단 노란 박스] 두 테이블의 쓰기는 **단일 트랜잭션**으로 묶입니다. 예를 들어 **match_results** INSERT는 성공했는데 **player_stats** UPSERT가 실패하면, 전적 집계는 누락되고 매치 기록만 남는 불일치가 생깁니다. 트랜잭션으로 묶어 둘 다 성공하거나 둘 다 취소되는 **원자성**을 보장합니다.

슬라이드 13 — MySQL 저장 흐름 🕒 약 65초

저장 프로세스는 총 7단계로 진행됩니다.

[→ 1번 카드] 먼저 매치가 종료되면 서버는 **MarkRoomEnded()** 를 호출하며 Ended 상태로 전환합니다.

[→ 2번 카드] 이어서 **BuildMatchResultPayload()** 로 저장할 결과 객체를 생성합니다.

[→ 3번 카드] 그 즉시 상태를 **Saving** 으로 바꾸고 클라이언트에 브로드캐스트합니다.

[→ 4번 카드] 백그라운드에서 **트랜잭션이 시작되어**, match_results INSERT와 player_stats UPSERT를 차례로 수행합니다.

[→ 5번 카드] 성공하면 COMMIT, 실패 시 **최대 3회 재시도** 후 롤백합니다.

[→ 6번 카드] 저장이 완료되면 **상위 10명 리더보드**를 best_score 내림차순으로 쿼리합니다.

[→ 7번 카드] 마지막으로 **ResultsReady** 상태로 전환하여 클라이언트에 결과 화면을 띄워줍니다.

슬라이드 14 — Unity 클라이언트 구조 🕒 약 45초

클라이언트는 **4개 계층**으로 깔끔하게 분리했습니다.

[→ 계층 구조 위에서 아래로] 가장 위의 **NetworkSpikeBootstrap** 은 씬이 로드될 때 필요한 컴포넌트들을 자동으로 생성하며, DontDestroyOnLoad로 씬 전환 시에도 유지됩니다.

그 아래 **NetworkSpikeApp** 이 게임 매니저로서 룸 진입, 종료, 그리고 서버 상태를 Unity 오브젝트에 반영하는 역할을 맡습니다.

NetworkSpikeClient 는 실제 TCP 연결을 담당하며, 백그라운드 ReadLoop와 2초마다 동작하는 HeartbeatLoop를 돌립니다. 서버에서 온 데이터는 **SynchronizationContext** 를 통해 안전하게 메인 스레드로 디스패치됩니다.

최하단에는 공유 DTO인 **NetworkSpikeProtocol** 이 서버와 동일한 메시지 구조를 정의합니다.

슬라이드 15 — 정리 & 데모 동선 🕒 약 30초

정리하자면, 이번 프로젝트는 **권위 서버 구조**로 로직의 보안성을 높이고, **TCP + Length-Prefix JSON** 으로 신뢰성 있는 통신을 구현했습니다. **6단계 룸 상태 머신**과 **MySQL 트랜잭션 3회 재시도 로직**을 통해 안정적인 전적 관리 시스템도 갖추었습니다.

[→ 우측 데모 동선] 이제 실제 구동 모습을 보여드리겠습니다. 서버 실행 후 두 클라이언트가 접속하고, 배터리를 획득하며 SlowShot을 사용하는 게임플레이를 시연한 뒤, 종료 후 MySQL에 match_results와 player_stats가 정상적으로 저장되는 것을 확인하겠습니다. **이상으로 발표를 마치겠습니다. 감사합니다.**